



CS 318 – Architecture and Organization

FINAL PROJECT/EXAM (COMPUTER PROGRAM PROJECT)

GROUP NO: 5

SECTION: BSCS 3-A

GROUP MEMBERS: BACSAIN, JAVE A.
 BONGALOS, BLESSIE FAITH S.
 ORTINERO, FREDERICK D.
 RICAFORT, DIVINO AL D.

ABSTRACT

Overview

The **Employee Record Tracker System** is a console-based database management application written in x86 Assembly language (using NASM in 32-bit). It allows complete management of employee information. This project showcases the actual use of low-level programming concepts, modular programming methods and data structures in assembly language.

Purpose

The entire purpose of this system is to give organizations a lightweight, efficient tool to manage their employee files with an easy to use but powerful command line interface. The application helps address the core requirements of needing to store, retrieve, and manipulate employee data from a database, while demonstrating the power and precision that comes with using assembly language programming for real world database applications.

Core Functionality

The system implements five essential operations through an intuitive menu-driven interface:

1. **Add Employee** - Users can register new employees by entering their name and what position they occupy. Multi-word entries are supported, and trailing whitespace is stripped automatically. The system includes a duplicate name check with confirmation prompts to avoid accidental duplicates.
2. **Delete Employee** - Users can remove employees by name or by position. The system natively handles cases of duplicate employee names and also automatically shrinks the array structure if there are one or more duplicates.



COLLEGE *of* COMPUTER STUDIES

3. **Search Employee** - Users can search for employees by name or by their position. Results are shown in a formatted table view, which includes labeled column headers and internally manages the numbering of the employee records displayed.
4. **Display Employees** - Users can review employee data in two views; complete and numbered comprehension of the full employee list, or one that groups employees by their position to help better visualize relationships between the data and quickly analyze general employee classifications.
5. **Exit** - Ensures clean program termination with proper resource cleanup.

Technical Implementation

The application leverages several advanced programming techniques:

- **Modular Architecture:** The application uses the cdecl calling convention by dividing the program into separate subroutines for each major functionality, which allows for code reusability and maintainability.
- **Memory Management:** This uses a static 32KB array structure, which can accommodate 500 employee records. Each employee record is 64 bytes (32 bytes for name and 32 bytes for position).
- **Input Validation:** It incorporates thorough error checking that checks the character limits enforced (with a 31 character maximum for each field), confirms the selections of the menu choices, prevents buffer overflows, and uses the stdin buffer to handle invalid inputs.
- **String Processing:** Custom string manipulation functions are provided to process input strings by removing new line characters and removing leading or trailing white space.
- **Array Processing:** During deletions the array compacts efficiently to eliminate gaps and consistently maintain the integrity of a sequential data structure.

Educational Importance

This project serves as a great and useful educational tool for understanding:

- Low-level memory management and direct hardware manipulation
- Assembly language syntax and instruction set architecture
- System call interfaces and C library integration



COLLEGE *of* COMPUTER STUDIES

- Data structure implementation at the assembly level
- Algorithm design for searching, sorting, and data manipulation
- Input/output operations and buffer management
- Modular programming and function calling conventions

Practical Applications

While developed as an academic project, the Employee Record Tracker demonstrates real-world applicability in:

- Small business employee management
- Department-level record keeping
- Educational demonstrations of database concepts
- Resource-constrained environments requiring minimal memory footprint
- Systems programming education and training
- Understanding foundational concepts before advancing to higher-level languages

Project Outcomes

The completed system successfully demonstrates:

- ✓ Full CRUD (Create, Read, Update, Delete) operations on employee data
- ✓ Robust ` and input validation mechanisms
- ✓ User-friendly interface with clear prompts and formatted output
- ✓ Scalability to handle up to 500 employee records
- ✓ Efficient memory utilization and data structure management
- ✓ Modular code organization following software engineering best practices

Conclusion

The Employee Record Tracker System illustrates how assembly programming can be used to build working and reliable business applications. This will provide the developer with valuable educational experiences in computer architecture, memory management, and concepts related to software development in general. It showcases assembly's precision and control while maintaining suitability for actual data management.



COLLEGE of COMPUTER STUDIES

SAMPLE RUN

YouTube link/s: <https://youtu.be/SO6xOV2YgAo>

Step 1

Download NASM and MinGW.

NASM link:

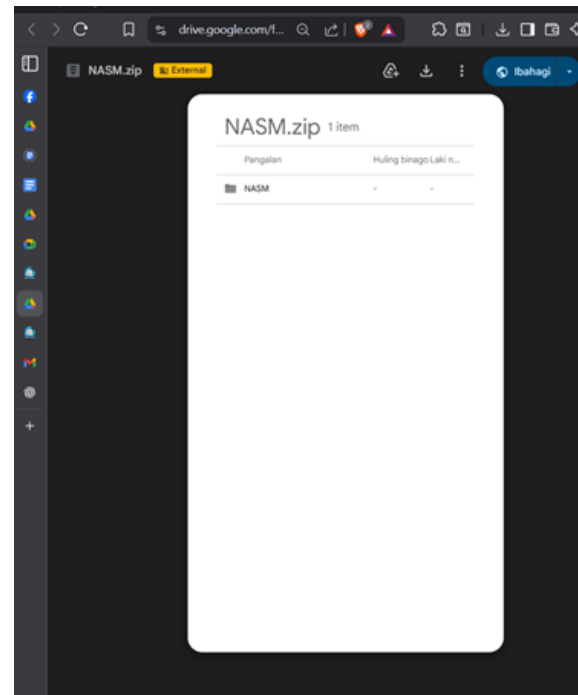
<https://drive.google.com/file/d/1CtUV63uubUHnIrhbhartGL2tyHibWq6/view?usp=sharing>

MinGW link:

<https://drive.google.com/file/d/10alhYZBE3QITYtOvcZtmhsdxi8ZeY1hK/view?usp=sharing>

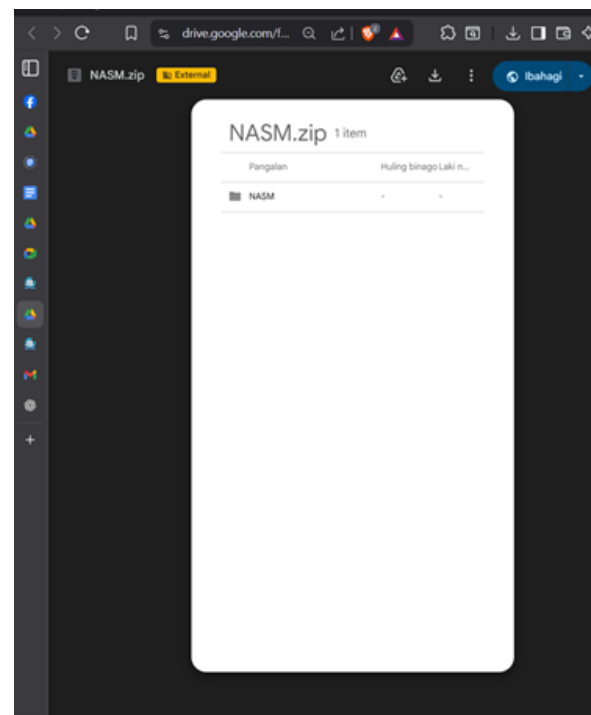
(These are needed for running the assembly program.)

Image 1



Step 2

1. Extract the installer files in your C drive.
2. Go to Control Panel -> Edit System Environment Variables -> Environment Variables.
 - Under User Variables, Edit Path
 - Click New and add the following:
 - C:\MinGW\bin
 - C:\NASM
3. Now, let's check if it is properly installed.
 - Open CMD





COLLEGE of COMPUTER STUDIES

- Type 'nasm --version', then type 'gcc --version' - This should give you the version installed in your system	
Step 3 Make an .asm file so that you can put your code there, just like what you can see in the image.	
Step 4 After that, run this code on your command prompt in the directory where your .asm file is saved: nasm -f win32 yourfilename.asm -o yourfilename.obj	
Step 6 We can now run the executable file by typing its name with its extension, for example: <i>yourfilename.exe</i> This command runs the program. Just make sure that the directory in the command prompt where you run this command is exactly where the file is.	



COLLEGE of COMPUTER STUDIES

Step 7

You chose option 1 to add an employee:

The program prompts you to enter the employee's name (maximum 31 characters). After entering the name, it prompts for the position.

- **Test case 1 (Valid Input):** The program enforces a 31-character limit. If the input exceeds this limit, it displays an error message and re-prompts for input.
- **Test case 2 (Input Exceeds 31 Characters)**
- **Test case 3 (Duplicate Name - Cancel):** When you attempt to add an employee with a name that already exists in the database, the program displays a warning message and asks for confirmation.
- **Test case 4 (Duplicate Name - Confirm)**

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 1

Enter employee name (max 31 chars): Jose Rizal
Enter employee position (max 31 chars): Machine Learning Engineer
Employee added successfully!
```

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 1

Enter employee name (max 31 chars): ThisIsAnExtremelyLongNameThatExceedsThirtyOneCharactersInLength
Error: Input exceeds 31 characters. Please try again.
```

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 1

Enter employee name (max 31 chars): Jose

WARNING: An employee with name 'Jose' already exists!
Do you want to add anyway? (y/n): n
Add operation cancelled.
```

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 1

Enter employee name (max 31 chars): Jose

WARNING: An employee with name 'Jose' already exists!
Do you want to add anyway? (y/n): y
Enter employee position (max 31 chars): Arki
Employee added successfully!
```



COLLEGE of COMPUTER STUDIES

Step 8

Add Multiple Employees (Minimum 10) and Display All Employees

You chose option 1 repeatedly to add at least 10 employees. Add the required minimum of 10 employees with various names and positions. The program successfully stores each entry.

- Choose option 4 to display employees, then option 1 to display all (the program displays a formatted table with employee numbers, names, and positions. All 10+ employees are listed).
- Choose option 4, then option 2 to display a grouped view (the program groups employees by their position, displaying a header for each position followed by the employees in that category).

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 4

=====
DISPLAY EMPLOYEES MENU
=====
1. Display All Employees
2. Display Employees by Position (Grouped)

Enter choice (1-2): 1

=====
NO.   NAME                POSITION
=====
1  Jose Rizal             Machine Learning Engineer
2  Jose                   Arki
3  Jose                   Arki
4  Jane Doe               Project Manager
5  Mike Johnson           Software Engineer
6  Sarah Williams         HR Manager
7  David Brown            Data Analyst
8  Emily Davis            Software Engineer
9  Robert Miller          Marketing Director
10 Lisa Wilson            Data Analyst
11 James Moore           Sales Representative
12 Maria Garcia          Software Engineer

Press Enter to continue...
```

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 4

=====
DISPLAY EMPLOYEES MENU
=====
1. Display All Employees
2. Display Employees by Position (Grouped)

Enter choice (1-2): 2

=====
EMPLOYEES GROUPED BY POSITION
=====

Position: Machine Learning Engineer
-----
1  Jose Rizal

Position: Arki
-----
2  Jose
3  Jose

Position: Project Manager
-----
4  Jane Doe

Position: Software Engineer
-----
5  Mike Johnson
8  Emily Davis
12 Maria Garcia

Position: HR Manager
-----
6  Sarah Williams

Position: Data Analyst
-----
7  David Brown
10 Lisa Wilson

Position: Marketing Director
-----
9  Robert Miller

Position: Sales Representative
-----
11 James Moore

Press Enter to continue...|
```



Step 9

Search Employee by Name

You chose option 3 to search, then option 1 to search by name: The program prompts for a name and displays all employees matching that name in a formatted table.

- Test case 1 (Name Found)
- Test case 2 (Name Not Found)

Search Employee by Position

You chose option 3 to search, then option 2 to search by position: The program prompts for a position and displays all employees with that position.

- Test case 1 (Position Found - Multiple Matches)
- Test case 2 (Position Found - Single Match)

Search Employee by Name

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 3

=====
SEARCH EMPLOYEE MENU
=====
1. Search by Name
2. Search by Position

Enter choice (1-2): 1

Enter name to search: James Moore

=====
SEARCH RESULTS
=====
NO.  NAME                POSITION
=====
11  James Moore          Sales Representative

Press Enter to continue...
```

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 3

=====
SEARCH EMPLOYEE MENU
=====
1. Search by Name
2. Search by Position

Enter choice (1-2): 1

Enter name to search: Blessie Faith
No employees found.

Press Enter to continue...|
```

Search Employee by Position

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 3

=====
SEARCH EMPLOYEE MENU
=====
1. Search by Name
2. Search by Position

Enter choice (1-2): 2

Enter position to search: Data Analyst

=====
SEARCH RESULTS
=====
NO.  NAME                POSITION
=====
7   David Brown          Data Analyst
10  Lisa Wilson           Data Analyst

Press Enter to continue...|
```




COLLEGE of COMPUTER STUDIES

	<pre>===== EMPLOYEE RECORD TRACKER SYSTEM ===== 1. Add Employee 2. Delete Employee 3. Search Employee 4. Display Employees 5. Exit Enter choice (1-5): 3 ===== SEARCH EMPLOYEE MENU ===== 1. Search by Name 2. Search by Position Enter choice (1-2): 2 Enter position to search: Assembly Coderist No employees found. Press Enter to continue... </pre>
Step 10 Delete Employee by Name You chose option 2 to delete, then option 1 to delete by name: The program prompts for a name and deletes all employees with that exact name. The array is automatically compacted. • Test case 1 (Name Found and Deleted) • Test case 2 (Name Not Found) Delete Employee by Position You chose option 2 to delete, then option 2 to delete by position: The program prompts for a position and deletes all employees with that exact position. Multiple employees can be deleted at once. • Test case 1 (Position Found - Multiple Deletions)	<pre>===== EMPLOYEE RECORD TRACKER SYSTEM ===== 1. Add Employee 2. Delete Employee 3. Search Employee 4. Display Employees 5. Exit Enter choice (1-5): 2 ===== DELETE EMPLOYEE MENU ===== 1. Delete by Name 2. Delete by Position Enter choice (1-2): 1 Enter name to delete: Jose Employee(s) deleted successfully! Press Enter to continue...</pre> <pre>===== EMPLOYEE RECORD TRACKER SYSTEM ===== 1. Add Employee 2. Delete Employee 3. Search Employee 4. Display Employees 5. Exit Enter choice (1-5): 2 ===== DELETE EMPLOYEE MENU ===== 1. Delete by Name 2. Delete by Position Enter choice (1-2): 1 Enter name to delete: Blessie No employees deleted. Press Enter to continue...</pre> <pre>===== EMPLOYEE RECORD TRACKER SYSTEM ===== 1. Add Employee 2. Delete Employee 3. Search Employee 4. Display Employees 5. Exit Enter choice (1-5): 2 ===== DELETE EMPLOYEE MENU ===== 1. Delete by Name 2. Delete by Position Enter choice (1-2): 2 Enter position to delete: Data Analyst Employee(s) deleted successfully! Press Enter to continue...</pre>



Verify deletion by Displaying all Employees

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 4

=====
DISPLAY EMPLOYEES MENU
=====
1. Display All Employees
2. Display Employees by Position (Grouped)

Enter choice (1-2): 1

=====
NO.  NAME                                POSITION
=====
1   Jose Rizal                          Machine Learning Engineer
2   Jane Doe                            Project Manager
3   Mike Johnson                        Software Engineer
4   Sarah Williams                      HR Manager
5   Emily Davis                         Software Engineer
6   Robert Miller                      Marketing Director
7   James Moore                        Sales Representative
8   Maria Garcia                       Software Engineer

Press Enter to continue...
```

Step 11
Test Whitespace Trimming

You chose option 1 to add an employee:
The program automatically trims leading and trailing whitespace from both name and position inputs.

Test case (Input with Extra Spaces) then verify

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 1

Enter employee name (max 31 chars):   Alice Cooper
Enter employee position (max 31 chars): Marketing Director
Employee added successfully!

=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 3

=====
SEARCH EMPLOYEE MENU
=====
1. Search by Name
2. Search by Position

Enter choice (1-2): 1

Enter name to search: Alice Cooper

=====
SEARCH RESULTS
=====
NO.  NAME                                POSITION
=====
9   Alice Cooper                          Marketing Director

Press Enter to continue...
```

Step 12
Test Invalid Menu Choices

You enter invalid choices at the main menu: The program validates menu input

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 0
Invalid choice, please try again.
```



COLLEGE of COMPUTER STUDIES

and displays an error message for invalid choices, then re-displays the menu.

- Test case 1 (Out of Range - Too Low)
- Test case 2 (Out of Range - Too High)
- Test case 3 (Non-numeric Input)
- Test case 4 (Invalid in Submenu)

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 6
Invalid choice, please try again.
```

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): abc
Invalid choice, please try again.
```

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 4

=====
DISPLAY EMPLOYEES MENU
=====
1. Display All Employees
2. Display Employees by Position (Grouped)

Enter choice (1-2): 5
Invalid choice, please try again.
```

Step 13

Test Empty Database Operations

Starting with a fresh program (0 employees): The program handles operations on an empty database gracefully with appropriate messages.

- Test case 1 (Display All - Empty)
- Test case 2 (Delete - Empty)
- Test case 3 (Search - Empty)

```
=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 4
No employees in database.

Press Enter to continue...

=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 2
No employees in database.

Press Enter to continue...

=====
EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit

Enter choice (1-5): 3
No employees in database.

Press Enter to continue...
```



COLLEGE *of* COMPUTER STUDIES

Step 14

You chose option 5 to exit:

The program displays an exit message and terminates cleanly, returning to the command prompt.

```
=====
                        EMPLOYEE RECORD TRACKER SYSTEM
=====
1. Add Employee
2. Delete Employee
3. Search Employee
4. Display Employees
5. Exit
Enter choice (1-5): 5
Exiting program.
```



COLLEGE of COMPUTER STUDIES

PROGRAM CODE

```
; =====  
; Employee Record Tracker  
;  
; GROUP MEMBERS:  
;  
; BACSAIN, JAVE A.  
; BONGALOS, BLESSIE FAITH S.  
; ORTINERO, FREDERICK D.  
; RICAFORT, DIVINO AL D.  
;  
; =====  
section .data  
    EOF equ -1  
    MAX_EMPLOYEES equ 500 ; we can change the employee limit to whatever, as  
long as we also change (employees resb) 32000 == (500)64  
    EMPLOYEE_SIZE equ 64  
    NAME_SIZE equ 32  
  
    fmt_int db "%d", 0  
    fmt_str db " %31[^\n]", 0  
    fmt_display db "Name: %s - Position: %s", 10, 0  
    format_employee db " %-3d %-30s %-30s", 10, 0  
    format_employee_grouped db " %-3d %-30s", 10, 0  
  
    menu_title db  
10, "=====", 10  
        db "                EMPLOYEE RECORD TRACKER SYSTEM", 10  
        db  
"===== ", 10, 0  
    menu_opt1 db "1. Add Employee", 10, 0  
    menu_opt2 db "2. Delete Employee", 10, 0  
    menu_opt3 db "3. Search Employee", 10, 0  
    menu_opt4 db "4. Display Employees", 10, 0  
    menu_opt5 db "5. Exit", 10, 0  
    menu_prompt db 10, "Enter choice (1-5): ", 0  
  
    display_title db  
10, "===== ", 10  
        db "                DISPLAY EMPLOYEES MENU", 10  
        db  
"===== ", 10, 0  
    display_opt1 db "1. Display ALL Employees", 10, 0
```



COLLEGE of COMPUTER STUDIES

```
display_opt2 db "2. Display Employees by Position (Grouped)",10,0
display_prompt db 10,"Enter choice (1-2): ",0

name_prompt db 10,"Enter employee name (max 31 chars): ", 0
pos_prompt db "Enter employee position (max 31 chars): ", 0
db_full_msg db "Employee database is full (maximum 500 employees).", 10, 0
success_msg db "Employee added successfully!", 10, 0
press_enter db 10, "Press Enter to continue...", 0
exit_msg db "Exiting program.", 10, 0
not_impl db "Feature not implemented yet.", 10, 0
invalid_msg db "Invalid choice, please try again.", 10, 0

search_title db
10,"=====",10
db " SEARCH EMPLOYEE MENU",10
db
"=====",10,0

search_opt1 db "1. Search by Name",10,0
search_opt2 db "2. Search by Position",10,0
search_prompt db 10,"Enter choice (1-2): ",0

search_name_prompt db 10,"Enter name to search: ", 0
search_pos_prompt db 10,"Enter position to search: ", 0
not_found_msg db "No employees found.", 10, 0
found_msg db "Found employee(s):", 10, 0

; Display headers
header_all db
10,"=====",10
db " NO. NAME POSITION",10
db
"=====",10,0

header_search db
10,"=====",10
db " SEARCH RESULTS",10
db
"=====",10
db " NO. NAME POSITION",10
db
"=====",10,0

header_grouped db
10,"=====",10
db " EMPLOYEES GROUPED BY POSITION",10
```



COLLEGE of COMPUTER STUDIES

```
db
"=====",10,0
display_pos_header db 10,"Position: %s",10
db
"-----",10,0
no_emp_msg db "No employees in database.", 10, 0

; Delete messages
delete_title db
10,"=====",10
db "DELETE EMPLOYEE MENU",10
db
"=====",10,0
delete_opt1 db "1. Delete by Name",10,0
delete_opt2 db "2. Delete by Position",10,0
delete_prompt db 10,"Enter choice (1-2): ",0

delete_name_prompt db 10,"Enter name to delete: ", 0
delete_pos_prompt db 10,"Enter position to delete: ", 0
deleted_msg db "Employee(s) deleted successfully!", 10, 0
not_deleted_msg db "No employees deleted.", 10, 0

; Duplicate warning messages
dup_warning db 10,"WARNING: An employee with name '%s' already
exists!",10,0
dup_prompt db "Do you want to add anyway? (y/n): ",0
dup_cancelled db "Add operation cancelled.",10,0

; Input validation messages
input_too_long db "Error: Input exceeds 31 characters. Please try
again.",10,0
fmt_char db " %c",0

; File mode for fdopen
mode_read db "r", 0

section .bss
employees resb 32000 ; 500 * 64 = 32000 bytes
count resd 1
buffer resb 64
choice resd 1
found_flag resd 1 ; flag to track if any match found
user_char resb 1 ; for y/n input
temp_buffer resb 64 ; temporary buffer for position comparison
```



```
position_printed resb 32 ; track which positions we've printed

; Storage for stdin handle
stdin_handle resd 1

section .text
    global _main
    extern _printf, _scanf, _strcpy, _getchar, _strcmp, _fgets, _fdopen,
    _strlen

; -----
; remove_newline: remove trailing newline from string
; -----
remove_newline:
    push ebp
    mov ebp, esp
    mov esi, [ebp+8]
    push esi
    call _strlen
    add esp, 4
    cmp eax, 0
    jz .done
    dec eax
    add esi, eax
    cmp byte [esi], 0Ah
    jne .done
    mov byte [esi], 0
.done:
    mov esp, ebp
    pop ebp
    ret

; -----
; trim_string: remove leading and trailing spaces
; -----
trim_string:
    push ebp
    mov ebp, esp
    push esi
    push edi
    push ebx

    mov esi, [ebp+8] ; source string
```




```
    ; Skip Leading spaces
.skip_leading:
    mov al, [esi]
    cmp al, 0
    je .empty_or_all_spaces
    cmp al, 32 ; space
    jne .found_start
    inc esi
    jmp .skip_leading

.found_start:
    ; Find end of string
    mov edi, esi
    xor ebx, ebx ; ebx = length counter
.find_end:
    mov al, [edi]
    cmp al, 0
    je .trim_trailing
    inc edi
    inc ebx
    jmp .find_end

.trim_trailing:
    ; edi points to null terminator, go back to find last non-space
    test ebx, ebx
    jz .empty_or_all_spaces
    dec edi
.remove_trailing:
    cmp edi, esi
    jb .done_trimming
    mov al, [edi]
    cmp al, 32
    jne .done_trimming
    mov byte [edi], 0
    dec edi
    jmp .remove_trailing

.done_trimming:
    ; Now shift the trimmed string to the beginning if needed
    mov edi, [ebp+8] ; destination (original pointer)
    cmp esi, edi
    je .already_at_start
.shift_loop:
    mov al, [esi]
```



```
    mov [edi], al
    test al, al
    jz .already_at_start
    inc esi
    inc edi
    jmp .shift_loop

.empty_or_all_spaces:
    mov edi, [ebp+8]
    mov byte [edi], 0

.already_at_start:
    pop ebx
    pop edi
    pop esi
    mov esp, ebp
    pop ebp
    ret

; -----
; clear_stdin_buffer: flush remaining input until newline
; -----
clear_stdin_buffer:
    push eax
.loop:
    call _getchar
    cmp eax, 10
    je .done
    cmp eax, EOF
    je .done
    jmp .loop
.done:
    pop eax
    ret

; -----
; _main: program entry point
; -----
_main:
    push ebp
    mov ebp, esp

    ; Initialize STDIN Handle dynamically
    push mode_read    ; "r"
```



COLLEGE of COMPUTER STUDIES

```
push 0 ; File Descriptor 0 (Standard Input)
call _fdopen
add esp, 8
mov [stdin_handle], eax ; Store the FILE* pointer

mov dword [count], 0

menu:
push menu_title
call _printf
add esp, 4

push menu_opt1
call _printf
add esp, 4

push menu_opt2
call _printf
add esp, 4

push menu_opt3
call _printf
add esp, 4

push menu_opt4
call _printf
add esp, 4

push menu_opt5
call _printf
add esp, 4

push menu_prompt
call _printf
add esp, 4

push choice
push fmt_int
call _scanf
add esp, 8

; Check if scanf succeeded (returns number of items read)
cmp eax, 1
jne .invalid_input
```



```
    call clear_stdin_buffer

    mov eax, [choice]
    cmp eax, 1
    je add_emp
    cmp eax, 2
    je delete_emp
    cmp eax, 3
    je search_emp
    cmp eax, 4
    je display_menu_handler
    cmp eax, 5
    je exit

; If we get here, choice is out of range
push invalid_msg
call _printf
add esp, 4
jmp menu

.invalid_input:
    call clear_stdin_buffer
    push invalid_msg
    call _printf
    add esp, 4
    jmp menu

delete_emp:
    push ebp
    mov ebp, esp

; Check if database is empty
    mov eax, [count]
    cmp eax, 0
    je .no_employees

    push delete_title
    call _printf
    add esp, 4

    push delete_opt1
    call _printf
    add esp, 4
```



```
push delete_opt2
call _printf
add esp, 4

push delete_prompt
call _printf
add esp, 4

push choice
push fmt_int
call _scanf
add esp, 8

; Check if scanf succeeded
cmp eax, 1
jne .invalid_input

call clear_stdin_buffer

mov eax, [choice]
cmp eax, 1
je .delete_by_name
cmp eax, 2
je .delete_by_position

; If we get here, choice is out of range
push invalid_msg
call _printf
add esp, 4
jmp .end

.invalid_input:
call clear_stdin_buffer
push invalid_msg
call _printf
add esp, 4
jmp .end

.delete_by_name:
push delete_name_prompt
call _printf
add esp, 4
```



```
push dword [stdin_handle]
push dword 64
push buffer
call _fgets
add esp, 12

push buffer
call remove_newline
add esp, 4

push buffer
call trim_string
add esp, 4

mov dword [found_flag], 0
mov ecx, [count]
xor esi, esi

.loop_name:
    cmp esi, ecx
    jge .check_deleted

    push ecx
    push esi

    ; Get employee name address
    mov eax, esi
    imul eax, EMPLOYEE_SIZE
    lea edi, [employees]
    add edi, eax

    ; Compare strings
    push buffer
    push edi
    call _strcmp
    add esp, 8

    pop esi
    pop ecx

    ; If strcmp returns 0, strings match - delete this employee
    cmp eax, 0
    jne .next_name
```



```
; Mark that we found a match
mov dword [found_flag], 1

; Shift all subsequent employees down by one slot
push ecx
push esi

mov edx, esi
.shift_loop_name:
    inc edx
    cmp edx, ecx
    jge .shift_done_name

    push ecx
    push edx

    ; Calculate source address (employee at index edx)
    mov eax, edx
    imul eax, EMPLOYEE_SIZE
    lea edi, [employees]
    add edi, eax

    ; Calculate destination address (employee at index edx-1)
    mov eax, edx
    dec eax
    imul eax, EMPLOYEE_SIZE
    lea ebx, [employees]
    add ebx, eax

    ; Copy employee (64 bytes)
    mov ecx, EMPLOYEE_SIZE
.copy_loop_name:
    mov al, [edi]
    mov [ebx], al
    inc edi
    inc ebx
    loop .copy_loop_name

    pop edx
    pop ecx
    jmp .shift_loop_name

.shift_done_name:
    pop esi
```



COLLEGE of COMPUTER STUDIES

```
pop ecx

; Decrement count
dec dword [count]
dec ecx

; Don't increment esi - check same position again
jmp .Loop_name

.next_name:
inc esi
jmp .Loop_name

.delete_by_position:
push delete_pos_prompt
call _printf
add esp, 4

push dword [stdin_handle]
push dword 64
push buffer
call _fgets
add esp, 12

push buffer
call remove_newline
add esp, 4

push buffer
call trim_string
add esp, 4

mov dword [found_flag], 0
mov ecx, [count]
xor esi, esi

.Loop_position:
cmp esi, ecx
jge .check_deleted

push ecx
push esi

; Get employee position address
```




```
mov eax, esi
imul eax, EMPLOYEE_SIZE
lea edi, [employees]
add edi, eax
add edi, NAME_SIZE

; Compare strings
push buffer
push edi
call _strcmp
add esp, 8

pop esi
pop ecx

; If strcmp returns 0, strings match - delete this employee
cmp eax, 0
jne .next_position

; Mark that we found a match
mov dword [found_flag], 1

; Shift all subsequent employees down by one slot
push ecx
push esi

mov edx, esi
.shift_loop_pos:
inc edx
cmp edx, ecx
jge .shift_done_pos

push ecx
push edx

; Calculate source address (employee at index edx)
mov eax, edx
imul eax, EMPLOYEE_SIZE
lea edi, [employees]
add edi, eax

; Calculate destination address (employee at index edx-1)
mov eax, edx
dec eax
```



```
    imul eax, EMPLOYEE_SIZE
    lea ebx, [employees]
    add ebx, eax

    ; Copy employee (64 bytes)
    mov ecx, EMPLOYEE_SIZE
.copy_loop_pos:
    mov al, [edi]
    mov [ebx], al
    inc edi
    inc ebx
    loop .copy_loop_pos

    pop edx
    pop ecx
    jmp .shift_loop_pos

.shift_done_pos:
    pop esi
    pop ecx

    ; Decrement count
    dec dword [count]
    dec ecx

    ; Don't increment esi - check same position again
    jmp .loop_position

.next_position:
    inc esi
    jmp .loop_position

.check_deleted:
    cmp dword [found_flag], 0
    jne .deleted
    push not_deleted_msg
    call _printf
    add esp, 4
    jmp .press_enter

.deleted:
    push deleted_msg
    call _printf
    add esp, 4
```



```
.press_enter:
    push press_enter
    call _printf
    add esp, 4
    call clear_stdin_buffer
    jmp .end

.no_employees:
    push no_emp_msg
    call _printf
    add esp, 4
    push press_enter
    call _printf
    add esp, 4
    call clear_stdin_buffer

.end:
    mov esp, ebp
    pop ebp
    jmp menu

search_emp:
    push ebp
    mov ebp, esp

    ; Check if database is empty
    mov eax, [count]
    cmp eax, 0
    je .no_employees

    push search_title
    call _printf
    add esp, 4

    push search_opt1
    call _printf
    add esp, 4

    push search_opt2
    call _printf
    add esp, 4

    push search_prompt
```



```
call _printf
add esp, 4

push choice
push fmt_int
call _scanf
add esp, 8

; Check if scanf succeeded
cmp eax, 1
jne .invalid_input

call clear_stdin_buffer

mov eax, [choice]
cmp eax, 1
je .search_by_name
cmp eax, 2
je .search_by_position

; If we get here, choice is out of range
push invalid_msg
call _printf
add esp, 4
jmp .end

.invalid_input:
call clear_stdin_buffer
push invalid_msg
call _printf
add esp, 4
jmp .end

.search_by_name:
push search_name_prompt
call _printf
add esp, 4

push dword [stdin_handle]
push dword 64
push buffer
call _fgets
add esp, 12
```



```
push buffer
call remove_newline
add esp, 4

push buffer
call trim_string
add esp, 4

mov dword [found_flag], 0
mov ecx, [count]
xor esi, esi

.loop_name:
    cmp esi, ecx
    jge .check_found

    push ecx
    push esi

    ; Get employee name address
    mov eax, esi
    imul eax, EMPLOYEE_SIZE
    lea edi, [employees]
    add edi, eax

    ; Compare strings
    push buffer
    push edi
    call _strcmp
    add esp, 8

    pop esi
    pop ecx

    ; If strcmp returns 0, strings match
    cmp eax, 0
    jne .next_name

    ; Print this employee if first match
    cmp dword [found_flag], 0
    jne .skip_header1

    push ecx
    push esi
```



```
    push header_search
    call _printf
    add esp, 4
    pop esi
    pop ecx
    mov dword [found_flag], 1

.skip_header1:
    push ecx
    push esi

    mov eax, esi
    imul eax, EMPLOYEE_SIZE
    lea edi, [employees]
    add edi, eax
    mov ebx, edi
    add ebx, NAME_SIZE

    push ebx
    push edi
    mov eax, esi
    inc eax
    push eax
    push format_employee
    call _printf
    add esp, 16

    pop esi
    pop ecx

.next_name:
    inc esi
    jmp .loop_name

.search_by_position:
    push search_pos_prompt
    call _printf
    add esp, 4

    push dword [stdin_handle]
    push dword 64
    push buffer
    call _fgets
    add esp, 12
```



```
push buffer
call remove_newline
add esp, 4

push buffer
call trim_string
add esp, 4

mov dword [found_flag], 0
mov ecx, [count]
xor esi, esi

.loop_position:
    cmp esi, ecx
    jge .check_found

    push ecx
    push esi

    ; Get employee position address
    mov eax, esi
    imul eax, EMPLOYEE_SIZE
    lea edi, [employees]
    add edi, eax
    add edi, NAME_SIZE

    ; Compare strings
    push buffer
    push edi
    call _strcmp
    add esp, 8

    pop esi
    pop ecx

    ; If strcmp returns 0, strings match
    cmp eax, 0
    jne .next_position

    ; Print this employee if first match
    cmp dword [found_flag], 0
    jne .skip_header2
```



```
    push ecx
    push esi
    push header_search
    call _printf
    add esp, 4
    pop esi
    pop ecx
    mov dword [found_flag], 1

.skip_header2:
    push ecx
    push esi

    mov eax, esi
    imul eax, EMPLOYEE_SIZE
    lea edi, [employees]
    add edi, eax
    mov ebx, edi
    add ebx, NAME_SIZE

    push ebx
    push edi
    mov eax, esi
    inc eax
    push eax
    push format_employee
    call _printf
    add esp, 16

    pop esi
    pop ecx

.next_position:
    inc esi
    jmp .loop_position

.check_found:
    cmp dword [found_flag], 0
    jne .press_enter
    push not_found_msg
    call _printf
    add esp, 4

.press_enter:
```




```
    push press_enter
    call _printf
    add esp, 4
    call clear_stdin_buffer
    jmp .end

.no_employees:
    push no_emp_msg
    call _printf
    add esp, 4
    push press_enter
    call _printf
    add esp, 4
    call clear_stdin_buffer

.end:
    mov esp, ebp
    pop ebp
    jmp menu

; -----
; add_emp: add a new employee
; -----
add_emp:
    push ebp
    mov ebp, esp

    ; Check if database is full
    mov eax, [count]
    cmp eax, MAX_EMPLOYEES
    jge .full

.get_name:
    ; Get name
    push name_prompt
    call _printf
    add esp, 4

    push dword [stdin_handle]
    push dword 64
    push buffer
    call _fgets
    add esp, 12
```



```
push buffer
call remove_newline
add esp, 4

push buffer
call trim_string
add esp, 4

; Check Length (max 31 chars)
push buffer
call _strlen
add esp, 4
cmp eax, 31
jg .name_too_long

; Check for duplicate name
mov ecx, [count]
xor esi, esi
.check_dup_loop:
cmp esi, ecx
jge .no_duplicate

push ecx
push esi

; Get employee name address
mov eax, esi
imul eax, EMPLOYEE_SIZE
lea edi, [employees]
add edi, eax

; Compare with input buffer
push buffer
push edi
call _strcmp
add esp, 8

pop esi
pop ecx

cmp eax, 0
je .duplicate_found

inc esi
```



```
    jmp .check_dup_loop

.duplicate_found:
    ; Show warning
    push buffer
    push dup_warning
    call _printf
    add esp, 8

    push dup_prompt
    call _printf
    add esp, 4

    ; Get confirmation (y or n)
    push user_char
    push fmt_char
    call _scanf
    add esp, 8

    call clear_stdin_buffer

    ; Check user's choice
    mov al, [user_char]
    cmp al, 'y'
    je .no_duplicate
    cmp al, 'Y'
    je .no_duplicate
    jmp .cancelled

.no_duplicate:

    ; Copy to employee array
    mov eax, [count]
    imul eax, EMPLOYEE_SIZE
    lea edi, [employees]
    add edi, eax

    push buffer
    push edi
    call _strcpy
    add esp, 8

.get_position:
    ; Get position
```



```
push pos_prompt
call _printf
add esp, 4

push dword [stdin_handle]
push dword 64
push buffer
call _fgets
add esp, 12

push buffer
call remove_newline
add esp, 4

push buffer
call trim_string
add esp, 4

; Check Length (max 31 chars)
push buffer
call _strlen
add esp, 4
cmp eax, 31
jg .position_too_Long

; Copy position (offset +32)
add edi, NAME_SIZE
push buffer
push edi
call _strcpy
add esp, 8

inc dword [count]

push success_msg
call _printf
add esp, 4

mov esp, ebp
pop ebp
jmp menu

.full:
push db_full_msg
```



```
    call _printf
    add esp, 4
    mov esp, ebp
    pop ebp
    jmp menu

.cancelled:
    push dup_cancelled
    call _printf
    add esp, 4
    mov esp, ebp
    pop ebp
    jmp menu

.name_too_long:
    push input_too_long
    call _printf
    add esp, 4
    jmp .get_name

.position_too_long:
    push input_too_long
    call _printf
    add esp, 4
    jmp .get_position

; -----
; display_menu_handler: handle display submenu
; -----

display_menu_handler:
    push ebp
    mov ebp, esp

    ; Check if database is empty
    mov eax, [count]
    cmp eax, 0
    je .no_employees

    push display_title
    call _printf
    add esp, 4

    push display_opt1
    call _printf
```



```
add esp, 4

push display_opt2
call _printf
add esp, 4

push display_prompt
call _printf
add esp, 4

push choice
push fmt_int
call _scanf
add esp, 8

; Check if scanf succeeded
cmp eax, 1
jne .invalid_input

call clear_stdin_buffer

mov eax, [choice]
cmp eax, 1
je .display_all
cmp eax, 2
je .display_grouped

; If we get here, choice is out of range
push invalid_msg
call _printf
add esp, 4
jmp .end

.invalid_input:
call clear_stdin_buffer
push invalid_msg
call _printf
add esp, 4
jmp .end

.display_grouped:
; Check if database is empty
mov ecx, [count]
cmp ecx, 0
```



```
je .no_employees

; Print grouped header
push header_grouped
call _printf
add esp, 4

; Outer loop: iterate through all employees
xor esi, esi
.outer_loop:
mov ecx, [count] ; Reload count each iteration
cmp esi, ecx
jge .press_enter

push ecx
push esi

; Get current employee's position
mov eax, esi
imul eax, EMPLOYEE_SIZE
lea edi, [employees]
add edi, eax
add edi, NAME_SIZE ; Point to position

; Copy position to temp_buffer
push edi
lea eax, [temp_buffer]
push eax
call _strcpy
add esp, 8

pop esi
pop ecx

; Check if we've already printed this position
; by searching backwards through employees 0 to esi-1
test esi, esi
jz .new_position

push ecx
push esi
xor edx, edx
.check_dup:
cmp edx, esi
```



```
jge .new_position_after_check

push ecx
push esi
push edx

mov eax, edx
imul eax, EMPLOYEE_SIZE
lea edi, [employees]
add edi, eax
add edi, NAME_SIZE

lea eax, [temp_buffer]
push eax
push edi
call _strcmp
add esp, 8

pop edx
pop esi
pop ecx

cmp eax, 0
je .skip_position ; Already printed this position

inc edx
jmp .check_dup

.new_position_after_check:
pop esi
pop ecx

.new_position:
; Print position header
push ecx
push esi

lea eax, [temp_buffer]
push eax
push display_pos_header
call _printf
add esp, 8

pop esi
```




```
pop ecx

; Inner loop: print all employees with this position
push ecx
push esi
xor edx, edx
.inner_loop:
mov ecx, [count] ; Reload count for comparison
cmp edx, ecx
jge .inner_done

push ecx
push esi
push edx

mov eax, edx
imul eax, EMPLOYEE_SIZE
lea edi, [employees]
add edi, eax
lea ebx, [edi + NAME_SIZE]

; Compare positions
lea eax, [temp_buffer]
push eax
push ebx
call _strcmp
add esp, 8

pop edx
pop esi
pop ecx

cmp eax, 0
jne .next_inner

; Print this employee with number (name only, position already in header)
push ecx
push esi
push edx

mov eax, edx
imul eax, EMPLOYEE_SIZE
lea edi, [employees]
add edi, eax
```



```
    push edi
    mov eax, edx
    inc eax
    push eax
    push format_employee_grouped
    call _printf
    add esp, 12

    pop edx
    pop esi
    pop ecx

.next_inner:
    inc edx
    jmp .inner_loop

.inner_done:
    pop esi
    pop ecx

.next_outer:
    inc esi
    jmp .outer_loop

.skip_position:
    pop esi
    pop ecx
    jmp .next_outer

.no_employees:
    push no_emp_msg
    call _printf
    add esp, 4
    jmp .press_enter

.display_all:
    mov ecx, [count]
    cmp ecx, 0
    je .press_enter

; Print display all header
push header_all
call _printf
```



COLLEGE of COMPUTER STUDIES

```
    add esp, 4

    xor esi, esi
.loop:
    mov ecx, [count] ; Reload count each iteration
    cmp esi, ecx
    jge .press_enter

    ; Calculate address
    mov eax, esi
    imul eax, EMPLOYEE_SIZE
    lea edi, [employees]
    add edi, eax

    ; Get position address
    mov ebx, edi
    add ebx, NAME_SIZE

    ; Print employee with number (printf corrupts registers!)
    push esi ; Save loop counter

    push ebx
    push edi
    mov eax, esi
    inc eax
    push eax
    push format_employee
    call _printf
    add esp, 16

    pop esi ; Restore loop counter

    inc esi
    jmp .loop

.press_enter:
    push press_enter
    call _printf
    add esp, 4
    call clear_stdin_buffer

.end:
    mov esp, ebp
    pop ebp
```



```
    jmp menu

; -----
; exit: clean exit
; -----
exit:
    push exit_msg
    call _printf
    add esp, 4
    xor eax, eax
    mov esp, ebp
    pop ebp
    ret
```



Republic of the Philippines
CAMARINES SUR POLYTECHNIC COLLEGES
Nabua, Camarines Sur



COLLEGE *of* COMPUTER STUDIES

GROUP ASSIGNMENT

NAME OF MEMBER	TASK ACCOMPLISHED
Bacsain, Jave A.	Code, Video
Bongalos, Blessie Faith S.	Testing and Code Documentation, Video
Ortinero Frederick D.	Video
Ricafort, Divino Al D.	Code, Video